PRACTICAL - 07

Aim: Form validator (SignupForm.java + Driver.java)

Code Screenshot:

SignupForm.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {}

public class SignupForm {
    @NotBlank
    private String username;

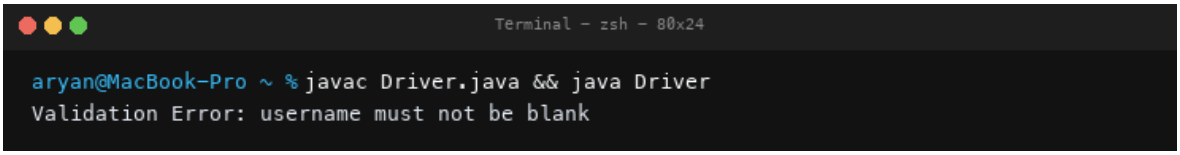
    public SignupForm(String u) { this.username = u; }
}
```

Driver.java

```java
import java.lang.reflect.Field;

public class Driver {
    public static void main(String[] args) throws Exception {
        SignupForm form = new SignupForm("   ");
        for (Field f : form.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            String val = (String) f.get(form);
            if (f.isAnnotationPresent(NotBlank.class) && val.trim().isEmpty()) {
                System.out.println("Validation Error: " + f.getName() + " must not be blank");
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to create custom annotations with @Retention and @Target.
- Understood Java Reflection API (Class, Field, getDeclaredFields).
- Learned to inspect and read field annotations dynamically at runtime.
- Implemented a declarative data-validation engine similar to Bean Validation.
- Handled IllegalAccessException through field.setAccessible(true).

Aim: Mini test runner (MathTests.java + Driver.java)

Code Screenshot:

MathTests.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {}

public class MathTests {
    @Run
    public void testAdd() { System.out.println("testAdd: PASSED"); }

    public void helper() { System.out.println("helper: should not run"); }

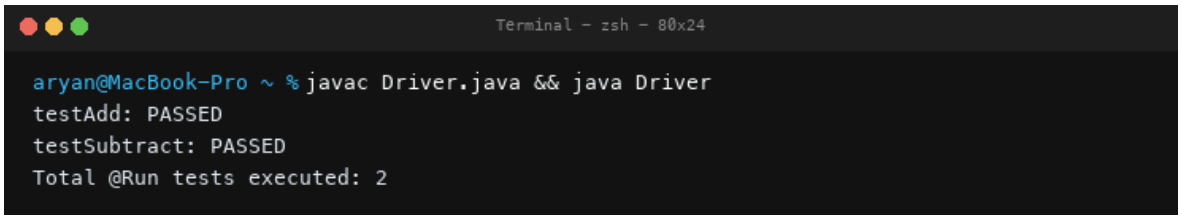
    @Run
    public void testSubtract() { System.out.println("testSubtract: PASSED"); }
}
```

Driver.java

```java
import java.lang.reflect.Method;

public class Driver {
    public static void main(String[] args) throws Exception {
        MathTests tests = new MathTests();
        int count = 0;
        for (Method m : MathTests.class.getDeclaredMethods()) {
            if (m.isAnnotationPresent(Run.class)) {
                m.invoke(tests);
                count++;
            }
        }
        System.out.println("Total @Run tests executed: " + count);
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood how testing frameworks like JUnit discover and run test methods.
- Learned method reflection (Method.invoke()) on runtime instances.
- Understood method filtering based on custom annotations.
- Implemented automated discovery and execution pipelines.
- Deepened appreciation for metadata-driven Java programming.